ПАСПОРТ ПРОЕКТА «GeoPoint»

Вид проекта	ИТ проект
Срок проекта	Дата начала: 12.10.2026.
Бюджет проекта	0 тыс. руб. без НДС (учебный проект, платные сервисы не используются)

1. **Описание проекта**

Предпосылки: Отзывы о заведениях на классических картах не всегда достоверны. Их можно оставить, не побывав на месте, или накрутить. Одновременно люди реже посещают местные заведения, из-за чего малый бизнес теряет выручку. Проект решает обе проблемы: даёт людям стимул выбираться из дома и обеспечивает достоверность отзывов.

Идея и подход: GeoPoint - веб-сервис, где пользователь отмечается в заведении, получает достижения за посещения и может оставить отзыв только после подтверждённого посещения. На основе достижений формируется рейтинг лучших посетителей. Подтверждение посещения выполняется по индивидуальному ключу заведения.

Реализация поэтапная: сначала база заведений нескольких улиц, затем карта с точками, затем отзывы и в последнюю очередь достижения.

Аналоги: Foursquare закрытое приложение, ориентированное на путешественников - берём механику отметок. Яндекс Карты - берём идею системы отзывов. Отличие GeoPoint: ориентация на жителей, которые хотят выбраться из дома, и отзывы только от реально посетивших.

Текущее состояние: Определена идея проекта и выбран технологический стек.

2. **Цель проекта**

Разработать и запустить в тестовую эксплуатацию веб-сервис GeoPoint, объединяющий людей через посещение местных заведений. С картой заведений не менее чем с трёх улиц, отметками посещений, отзывами только от посетивших и системой достижений.

3. **Выгоды и критерии успешности**

Выгоды: Пользователям - повод для живого общения и достоверные отзывы. Заведениям - приток посетителей. Финансовая выгода по завершении проекта не предусмотрена. В перспективе возможна реклама без влияния на рейтинг заведений.

4. **Объём проекта и результаты**

№	**Результаты проекта**
1.	ФИНАЛЬНЫЙ РЕЗУЛЬТАТ. Веб-сервис GeoPoint разработан и запущен в тестовую эксплуатацию

№	Результаты проекта
2	Создана база данных заведений
3	Реализована карта с точками заведений
4	Реализована система отзывов
5	Реализованы отметки по ключу заведения, достижения и рейтинг посетителей

5. Технологический стек

Компонент	Технология
База данных	PostgreSQL с расширением PostGIS (хранение координат и поиск заведений рядом)
Серверная часть и веб-интерфейс	Python, Django (GeoDjango для работы с картами), шаблоны Django
Карта	Leaflet и OpenStreetMap (бесплатно, без ключей API) или российские аналоги по типу RuMap или СДЭК ГИС
Подтверждение посещения	Индивидуальный ключ заведения или QR-кода; дополнительно проверка геопозиции средствами PostGIS
Авторизация	Встроенная система пользователей Django
Развёртывание	Docker Compose
Контроль версий	Git, GitHub (код и документация проекта)

Техническая часть

Требования к системе

Роли пользователей

1) Гость (неавторизованный пользователь): может просматривать интерактивную карту, список заведений, их рейтинг и читать отзывы.
2) Пользователь (Посетитель): может авторизовываться, искать заведения поблизости, отмечаться в заведениях (чекин) по секретному ключу или QR-коду и геопозиции, оставлять отзывы (только после успешного чекина), получать достижения и участвовать в рейтинге посетителей.
3) Администратор: управляет базой заведений (добавление точек, координат, описаний), генерирует и обновляет индивидуальные ключи(QR-коды) заведений, модерирует отзывы.
4) Владелец заведения: Дает пользователям индивидуальные ключи, отмечая их посещения в заведение.

Функциональные требования

Модуль авторизации и профиля:

1) Регистрация, вход и выход с использованием стандартной системы аутентификации Django.
2) Просмотр профиля пользователя: история посещений, полученные достижения, текущее место в рейтинге.

Модуль карты и заведений:

1) Отображение интерактивной карты с маркерами заведений.
2) Просмотр карточки заведения (название, адрес, среднее значение рейтинга, список подтверждённых отзывов).

Модуль подтверждения посещений (Check-in):

1) Ввод индивидуального ключа заведения или сканирование QR-кода (переход по уникальной ссылке чекина).

2) Ограничение на повторные отметки (например, не более одной проверки в одном заведении в сутки для защиты от накрутки).

Модуль отзывов и оценок:

1)Проверка права на отзыв: форма отправки отзыва и оценки (от 1 до 5 звёзд) доступна только при наличии подтверждённой проверки в данном заведении.

2)Один подтверждённый визит (или один пользователь) может оставить один актуальный отзыв на заведение.

Модуль достижений и рейтинга:

1)Автоматическая проверка условий и выдача достижений после каждой проверки.

2)Формирование таблицы лидеров по количеству посещений и набранным очкам достижений.

Архитектура

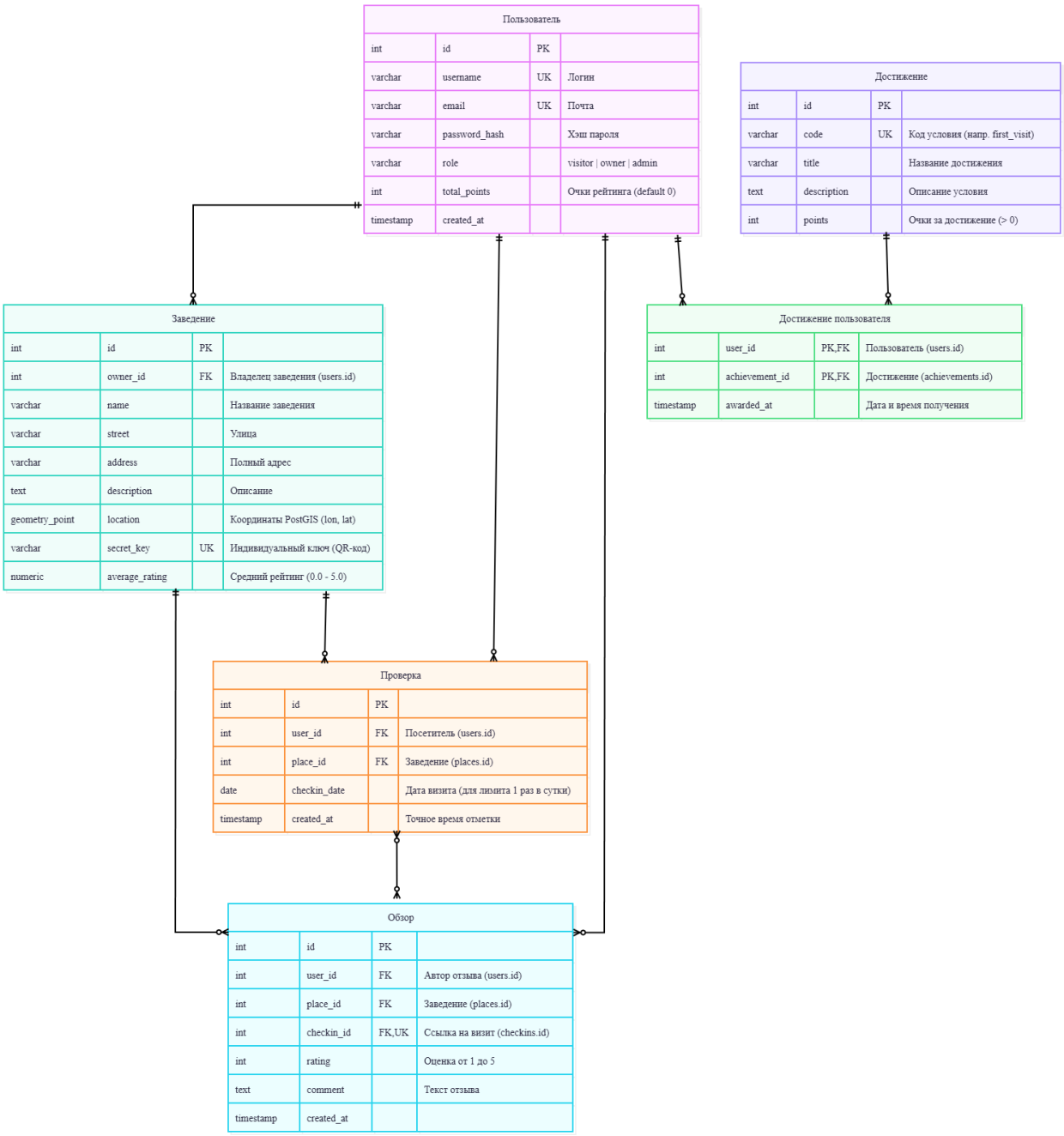
Клиентская часть (это всё, что работает в браузере) связывается с серверной частью, чтобы получить данные, авторизоваться, подтвердить посещения и отправить отзывы. Серверная часть держит секретные ключи заведений, проверяет, кто ты (Гость, Посетитель, Владелец или Администратор), следит за тем, чтобы ты не отмечался чаще одного раза в сутки, и даёт доступ к форме отзыва только после подтверждённого посещения.

Для карты используется библиотека Leaflet и тайлы OpenStreetMap (или российские аналоги, такие как RuMap, СДЭК ГИС). Адрес этих тайлов можно настроить, и он не влияет на работу сервера.

При запуске на локальной машине сервисы связываются по именам, PostgreSQL с расширением PostGIS работает на порту 5432 и не должен быть доступен снаружи. Браузер обращается к адресу веб-сервера Django, а для загрузки карты требуется HTTPS-соединение (порт 443) к серверу тайлов

OpenStreetMap или RuMap. Точный адрес приложения и порты (например, 8000) зависят от того, как ты настроил запуск.

ER-Диаграмма



Ограничения (в стиле документации твоего друга)

1) users.username и users.email уникальны;
2) users.role принимает одно из значений: visitor (посетитель), owner (владелец заведения), admin (администратор);

3) users.total_points — целое неотрицательное число (>= 0, по умолчанию 0);
4) places.secret_key уникален (индивидуальный ключ заведения для генерации QR-кода не может совпадать у разных точек);
5) places.owner_id ссылается на users.id (один владелец может управлять одним или несколькими заведениями);
6) не более одной отметки о посещении (checkins) на тройку user_id / place_id / checkin_date (защита от накрутки: один пользователь может отметиться в одном заведении не чаще одного раза в сутки);
7) один актуальный отзыв (reviews) на пару user_id / place_id (один пользователь может оставить только один отзыв на конкретное заведение);
8) reviews.checkin_id обязателен (NOT NULL), уникален и ссылается на существующую запись в checkins — оставить отзыв без подтверждённого посещения невозможно на уровне структуры БД;
9) reviews.rating — целое число от 1 до 5 (CHECK (rating >= 1 AND rating <= 5));
10) achievements.code уникален, а achievements.points — целое число больше нуля (CHECK (points > 0));
11) одно полученное достижение на пару user_id / achievement_id в таблице user_achievements (составной первичный ключ PRIMARY KEY (user_id, achievement_id) — одну и ту же награду нельзя получить дважды).

API и маршруты сервера

Базовый префикс для асинхронных запросов карты и клиентских скриптов: /api. Формат обмена данными: JSON (для точек на карте - GeoJSON). Защищённые методы требуют авторизации через встроенную систему сессий Django.

/api — логическая часть пути запроса к веб-серверу Django. Параметры подключения к базе данных PostgreSQL и внутренние секретные настройки сервера в браузер не передаются.

Авторизация

Plaintext

POST /auth/register

POST /auth/login

POST /auth/logout

GET /auth/me

Карта и заведения

Plaintext

GET /api/places?street=&category=&search=

GET /api/places/nearby?lat=...&lon=...&radius=1000

GET /api/places/{place_id}

POST /admin/places

Публичные методы возвращают список заведений, их координаты для отрисовки маркеров в Leaflet, адрес и средний рейтинг. Секретный индивидуальный ключ заведения (secret_key) в публичных ответах скрыт и никогда не передаётся обычным посетителям.

Подтверждение посещений (Check-in) и управление ключами

Plaintext

POST /api/places/{place_id}/checkin # {"secret_key": "A7X9K2"}

GET /api/checkin/qr/{secret_key} # Быстрая отметка по QR-ссылке

GET /api/owner/places/{place_id}/key # Просмотр текущего ключа и QR-кода

POST /api/owner/places/{place_id}/key/rotate # Генерация нового ключа заведения

Методы /api/owner/... доступны только владельцу данного заведения (owner) и администратору (admin). При успешном чекине сервер проверяет

совпадение ключа и убеждается, что пользователь ещё не отмечался в этом заведении за текущие сутки.

Отзывы и оценки

Plaintext

```
GET    /api/places/{place_id}/reviews
POST   /api/places/{place_id}/reviews       # {"rating": 5, "comment": "Отличное место!"}
PATCH  /api/reviews/{review_id}
DELETE /api/reviews/{review_id}
```

Отправка отзыва (POST) доступна только пользователям, у которых есть подтверждённый чекин в данном заведении. На одну пару пользователь/заведение допускается только один актуальный отзыв.

Пользователи, достижения и рейтинг

Plaintext

```
GET   /api/users/{username}
PATCH /api/users/me/profile
GET   /api/users/{username}/checkins
GET   /api/users/{username}/achievements
GET   /api/leaderboard?limit=20
```

После каждого успешного чекина сервер автоматически проверяет условия достижений, начисляет очки и обновляет позицию пользователя в таблице лидеров (/api/leaderboard).

Ошибки

JSON

```
{"detail": {"code": "CHECKIN_REQUIRED", "message": "Оставить отзыв можно только после подтверждённого посещения заведения"}}
```

Коды ответов: 400, 401, 403, 404, 409, 422.

1) 401 Unauthorized - попытка выполнить отметку или оставить отзыв без входа в аккаунт.
2) 403 Forbidden - попытка оставить отзыв без подтверждённого чекина (CHECKIN_REQUIRED) или попытка постороннего пользователя запросить секретный ключ заведения.
3) 409 Conflict — попытка отметиться в одном и том же заведении повторно в течение одних суток (ALREADY_CHECKED_IN_TODAY) или оставить второй отзыв на то же заведение.
4) 422 Unprocessable Entity - введён неверный индивидуальный ключ заведения (INVALID_SECRET_KEY) или указана оценка вне диапазона от 1 до 5.

Сервер проверяет права доступа и роль пользователя на каждое изменение данных. Индивидуальные ключи заведений хранятся и сверяются только на стороне сервера.